Forward Deployed Engineering from First Principles to Enterprise Delivery — Advanced

Portfolio projects · lessons 21–32

Each takes 1–3 days and ends with something to show an employer.

Project 1: Basket Bazaar: cut six stores over from proof of concept to production

Goal: Move the one-store delivery-issues proof of concept to six stores, one orders file per store, and roll a new column out to two running nodes with zero failed requests, the way a real customer cut-over runs.

Uses lessons: 05, 14, 16, 17, 21, 22, 23, 24, 24a

1. Write a production gap register with 12 rows from the proof of concept's own numbers (lesson 21); each row names an owner, a measured value and its source, and a check refuses any row with no source.

2. Generate a synthetic orders export of 2,700 rows across six stores (store-01 to store-06, 400 to 600 rows each, fixed seed 42) with 3% duplicate rows and missing, wrong and late item issues; split it into one CSV per store and check that the cleaned rows add back to the export's unique count.

3. Serve all six stores from one FastAPI service where every request is scoped to one store id, and add a test that asks for store-03 with a store-05 order id and expects HTTP 404 (lesson 23).

4. Run the triage service on two Docker Compose nodes behind an nginx proxy, then roll out version 2.0.0, which adds one column to the orders file, one node at a time while version 1.0.0 still reads the old file (lesson 24).

5. Drive 20 requests per second for 3 minutes during the rollout with a short load script; record non-2xx responses and p95 latency for the minute before, during and after the change (lesson 16 defines the latency panel); the target is zero non-2xx.

6. Trigger a wrong-order rollout on purpose, roll back to version 1.0.0 with one command, and time it from the first failed health check to the restored version; the target is under 2 minutes.

7. Run the nightly order batch with the lesson 24a pattern on the two Compose nodes: first a 1/100 scale of 12,000 orders (a 216-second window, 150 ms per mock customer call), with 9 calls in flight in total (1.2 million orders over 21,600 s at 150 ms needs 8.3, so 9 with headroom); partition by zlib.crc32 into 4 parts, upsert on (order_id, run_id), and write a manifest with the input's SHA-256 before the run starts; then kill one node at about half-way, restart it, and reconcile every part against the manifest; the targets are 0 duplicate rows and 0 missing rows, with 12,000 rows in total.

8. Write the handover pack for the customer's team (runbooks, an ownership matrix and a 30-minute training drill) and run the coverage check until it returns an empty list of gaps (lesson 22).

Deliverable: A Git repository with the six-store service, the two Compose files, the load script, the cut-over log, the batch run folder (manifest, per-part counts and reconciliation output) and the handover pack. The README has a table of measured numbers: orders per store, non-2xx count during the rollout, p95 latency before, during and after, rollback time, batch rows in, duplicate and missing rows after the node kill (target 0 and 0), the batch's measured run time against the 216-second window, and handover gaps (target 0). It also states the full-scale plan from the same arithmetic: 1.2 million orders at 9 calls in flight takes about 20,000 seconds (5.6 hours), inside the six-hour window.

Stretch: Replace the all-at-once rollout with a canary: send 10% of requests to version 2.0.0 for 5 minutes, and write the promote-or-roll-back rule in the README before the run. · Move the per-store daily order cap into the Excel workbook from lesson 23 and show that a cap change takes effect on the next request without a restart.

Project 2: Harbor Credit Union card-dispute intake: a security evidence pack

Goal: Build a card-dispute intake service for a regional credit union (a regulated, personal-data domain, using synthetic records only) and answer the bank's security questionnaire with evidence a reviewer can check, not with memory.

Uses lessons: 13, 15, 16, 17, 22, 27

1. Generate 300 synthetic dispute records with a fixed seed; put a fake name@example.com address, a 555-01xx phone number or a Luhn-invalid card-shaped number into 40 free-text notes, and check that no value matches a real person or a real card.

2. Write the intake API in FastAPI that stores each dispute with its notes redacted (the redaction function from lesson 27) and writes exactly one audit line per write with actor, action, record id and request id.

3. Create three service accounts (intake, agent, auditor) with scoped tokens over four endpoints, and add one test per account that asserts which endpoints it may call and gets 403 for the rest (lesson 13).

4. Run the service on a Compose network with internal: true and an egress proxy that allows one model endpoint, then prove that the service reaches the proxy and that a direct external connection fails (lesson 15).

5. Answer 20 questionnaire questions on residency, PII, audit, access and retention, where each answer links to a passing check, a log line or a config file; the pack script fails if any answer has no evidence behind it (lesson 27).

6. Measure PII in raw notes and in stored notes, and count audit lines per write; the targets are 0 PII hits in stored notes and exactly 1 audit line per write over 300 writes.

7. Write runbooks for three incidents (a duplicate dispute, a redaction miss and a leaked token), each with a verify step that a second engineer can run (lesson 17).

Deliverable: A Git repository with the intake service, the access tests, the Compose files, the questionnaire pack with its check script and the three runbooks. The README has a table of measured numbers: raw and stored PII hits, audit lines per write, number of questionnaire answers with a passing check (target 20 of 20) and the number of tests that pass.

Stretch: Add a model-based grader that looks for personal data the regex misses (lesson 12 grader patterns), and report its miss rate and the regex's miss rate on 100 held-out notes. · Add a retention job that runs in dry-run mode first, lists the records it would delete under the RETENTION_DAYS check from lesson 27, and writes each real deletion to the audit trail.

Project 3: Basket Bazaar unit economics and a reusable core: one customer's rules, one shared component

Goal: Turn a finished customer project into a product decision: price the triage service per store, find the levers that move the bill most, separate the customer's rules from the core code, and make the case to product with evidence.

Uses lessons: 18, 25, 26, 29, 30

1. Build a cost model that starts from the measured token counts in lesson 18's recorded calls and the price table from lesson 29, with the table's date printed next to every price.

2. Compute cost per triaged issue and cost per store per month for contract sizes of 1, 6 and 20 stores at 300 issues per store per month, and list at least five levers with their monthly saving.

3. Move the Store 03 18:00 closing rule into one config dict, keep clean_rows and totals_by_item as the core, and rerun the characterisation check that shows the 14 versus 45 egg count error and passes after the fix (lesson 25).

4. Write a field report from a request log: at least 6 logged requests from at least 2 customers, grouped by theme, with each number linked to its source line and labelled core or custom (lesson 26).

5. Write a business case for the top theme with the payback in months and the three-year net, and state the no-go threshold before the numbers are computed (lesson 30).

6. Write a one-page reply that declines a second feature in writing and offers a priced pilot in one store for a fixed number of working days, as lesson 30 does (10 working days).

Deliverable: A Git repository with the cost model, the refactored cleaner with its characterisation test, the field report, the business case and the reply. The README has a table of measured numbers: cost per issue, cost per store per month at each contract size, the saving of the top two levers, the characterisation result before and after (14 and 45 eggs), and the payback months and three-year net.

Stretch: Model a prompt-prefix cache on 60% of input tokens, as lesson 29's lever does, and route the easy cases to a smaller model; use the lesson 12 graders to check whether the quality holds at the lower cost. · Move the closing rule into a per-store file that a store manager can edit, and refuse any close time outside 06:00 to 23:59 with a clear error.

Project 4: Cold-chain logistics: a second customer on the same core, and a connector library

Goal: Deliver a second customer in a different domain (refrigerated trucks and temperature sensors) on the same core code, and extract only what the second customer proves is shared, the way a platform team does after the second deployment.

Uses lessons: 04, 05, 11, 21, 23, 25, 26

1. Generate a synthetic sensor export for 8 trucks over 30 days at one reading per 5 minutes (about 69,000 rows, fixed seed), with 2% duplicate rows, 1% out-of-range readings and two timestamp formats.

2. Write a connector for the sensor CSV that imports the cleaner from the Basket Bazaar core package and adds only the customer's truck ids and allowed range of -25 to -15 degrees C, and keep the custom code under 40 lines.

3. Write an eval set of 30 excursion windows (more than 2 degrees C for 20 minutes or more), labelled from the injected excursions and held out from tuning; grade the detector against the labels by counting true and false positives and negatives, then report precision and recall.

4. Run the shared contract tests from the Basket Bazaar project against the new connector, record how many pass, and move any failing rule into config rather than into the core.

5. Write a one-page extraction decision for each of three candidate shared parts (the cleaner, the paging client and the audit writer), using evidence from the second customer and not from guesses (lesson 25).

6. Write a field report with a platform backlog proposal, with three numbers each traced to a run (lesson 26).

Deliverable: A Git repository with two customer folders that share one core package, the shared contract tests, the eval set and its results, and the extraction decisions. The README has a table of measured numbers: rows in, rows dropped by reason, custom and core lines, detector precision and recall, and contract tests passing.

Stretch: Pin the core package version in each customer's requirements, make a breaking change in the core, and show that the pinned customer is unaffected while the other fails its tests. · Add a new sensor column to the export and roll out the connector change with the rolling pattern from lesson 24, with no run of the pipeline losing rows.

Project 5: Triage service under a provider outage: a failure drill with a recovery budget

Goal: Break the model provider on purpose under a burst of load, measure what customers would see, and show that the triage feature degrades to a labelled fallback with a capped retry budget and recovers on its own, the way an on-call team rehearses before go-live.

Uses lessons: 14, 16, 17, 18, 24, 28

1. Start the lesson 28 Compose stack and send one healthy triage request; confirm that the answer's source says model and that the call is counted once in the recording.

2. Write a load script that sends 200 messages from fixed seed 28 with 10 concurrent workers, and records for each answer its source (model or rules), its HTTP status and its latency in milliseconds.

3. Run three phases of 200 messages each: healthy, provider down (the route answers 503 to every call) and recovery after the breaker's reset timeout (set it to 10 s and write it in the README); record the fallback share, the p95 latency (nearest rank) and the non-2xx count for each phase.

4. Change the retry rule to one retry per call with a 0.5 to 0.75 s wait, and add a process-wide budget of 10 retries per minute shared by all workers (the platform decision lesson 28 leaves open); show that a 401 error is not retried and that total provider attempts stay under the budget during the outage.

5. Set alert thresholds on the fallback share and the p95 latency (lesson 16 panels), write the runbook entry for the alert (lesson 17), and show that the alert fires in the outage phase and clears in the recovery phase.

6. Write the drill report: a table with one row per phase and a go-live rule that names the fallback-share ceiling and the recovery time target (under 60 s to the first model answer after the route is healthy).

Deliverable: A Git repository with the load script, the retry and breaker settings, the alert rules, the runbook entry and the drill report. The README has a table of measured numbers per phase: messages, model answers, fallback answers, non-2xx count, p95 latency, provider attempts against the 10-per-minute budget, and recovery time to the first model answer.

Stretch: Add a second provider route behind the same fallback order, check it through the lesson 27 security questions (data flows, retention, subprocessors) and measure its answer quality on 30 messages with the lesson 12 graders. · Raise the load to 50 concurrent workers, find the worker count at which the p95 latency in the healthy phase passes 1 s, and explain in the README what limits it.